

Projet long – M1 IMPAIRS

Algorithme d'apprentissage supervisé pour
la détection d'attaque DoS sur un réseau

26 mai 2026

Contexte et Objectifs

- Contexte
 - L'ANSSI constate un regain d'attaques par DDoS
 - Les états et entreprises doivent se mettre en conformité : Cyber Resilience Act, NIS 2 et se protéger des attaques
- Objectifs
 - Construire un réseau de neurones simplifié dont l'apprentissage est supervisé
 - L'algorithme doit identifier des attaques et/ou des anomalies sur un réseau
- Critères de réussite
 - Comparaison à d'autres algorithmes existants supervisés et non-supervisés
 - Taux de précision du modèle

Description du projet : 3 scénarios d'utilisation

1. Audit post-mortem

Analyser une capture .csv après incident pour identifier les paquets suspects

→ liste de flux étiquetés 0/1.

L'algorithme indique pour chaque ligne, si elle présente une anomalie ou non

```
dt,switch,sourcelP,destinationIP,packetCount,byteCount,duration,duration_nsec,total  
_duration,flows, packetins,pktperflow,byteperflow,pktrate,Pairflow,Protocol,port_no,t  
x_bytes,rx_bytes,tx_kbps,rx_kbps,tot_kbps,label
```

Normale : sans anomalie

```
11425,1,10.0.0.1,10.0.0.8,45304,48294064,10  
0,716000000,1.01E+11,3,1943,13535,144283  
10,451,0,UDP,3,143928631,3917,0,0,0,0
```

Anormale : anomalie détectée

```
9936,5,10.0.0.18,10.0.0.7,48049,50067058,  
153,436000000,1.53E+11,3,1931,9405,9800  
010,313,0,UDP,3,3959,164816371,0,10298,1  
0298,1
```

Anomalie

Déviations par rapport à la baseline historique, telle que définie par la colonne 'label' de notre jeu de données.

Description du projet : 3 scénarios d'utilisation

2. Baseline de référence

Entraîner sur du trafic pour figer un comportement de référence, puis détecter toute déviation.

3. Comparaison d'algorithmes

Comparaison côte-à-côte de 3 méthodes (notre réseau de neurones maison, KNN, Isolation Forest) sur les mêmes données.

Vrai Positif

Anomalie
correctement
détectée

Vrai Négatif

Trafic normal classé
normal

Faux positif

Fausse alerte

Faux négatif

Situation critique —
attaque non
détectée

Pipeline d'exécution en 5 étapes

Vue d'ensemble



Orchestrateur — main.py

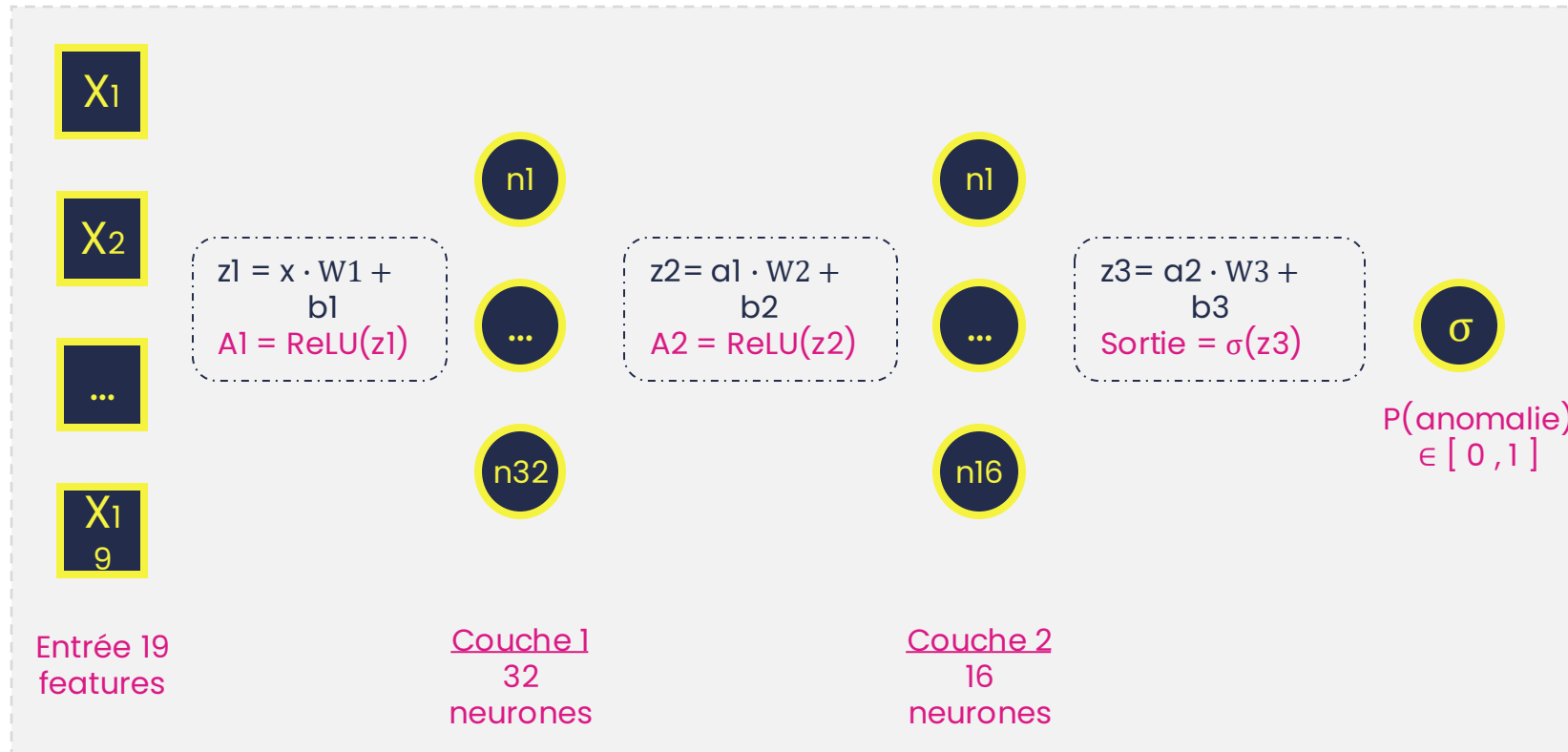
- Coordonne les 5 étapes
- Appelle run_evaluation()
- Produit rapport terminal + PNG

Modèles — interface polymorphique

- protoN.py — réseau de neurones maison
- knn_model.py — K plus proches voisins
- isol_forest_model.py — Isolation Forest

Architecture du réseau de neurones

2 couches cachées – ReLU –
1 couche de sortie – Sigmoid



Initialisation

Propagation

Perte et
rétropropagation

Mise à jour

Planning et organisation de l'équipe

Gestion de projet

	Ressources	Date de Début	Date de Fin
Lancement du projet		09-oct-25	11-déc-25
Recherche du sujet	Antoinette + Gabriel	09-oct	04-déc
Identification du sujet	Antoinette + Gabriel	09-oct	04-déc
Validation	Enseignants	04-déc	04-déc
Réalisation du planning	Antoinette	04-déc	11-déc
Développement principal		19-déc-25	27-mar-26
Développement du réseau à 8 neurones	Antoinette + Gabriel	19-déc	23-févr
Cadrage		19-déc	21-janv
Implémentation		21-janv	23-févr
Optimisation des poids du SNN avec la méthode de descente par gradient Adam	Antoinette	01-févr	27-mars
Cadrage		01-févr	15-févr
Implémentation		15-févr	27-mars
Développement intermédiaire		26-avr-26	21-mai-26
Comparaison des vitesses de convergence	Gabriel	26-avr	13-mai
Entre notre algorithme et l'algorithme KNN		26-avr	13-mai
Entre notre algorithme et l'algorithme de Isolation Forest		04-mai	13-mai
Débogage et validation	Antoinette + Gabriel	13-mai	21-mai
Tests		10-mai	21-mai
Comparaison d'efficacité et visual plots	Gabriel	10-mai	21-mai
Fonctions des comparaisons d'efficience entre algorithmes et plots graphiques	Gabriel	10-mai	21-mai
Rendu de projet		10-mai	21-mai
Réalisation et montage vidéo	Antoinette + Gabriel	10-mai	22-mai
Présentation powerpoint	Antoinette	22-mai	26-mai

Difficultés rencontrées

Vue d'ensemble

1. Appréhension des concepts mathématiques

- le calcul de la loss
- la descente de gradient
- l'implémentation d'Adam

2. Bug du seuil de décision

Au départ, le seuil de classification (sigmoïde > seuil) était calculé à partir de la perte (loss) — sans relation monotone avec la probabilité.

Conséquence : accuracy bloquée à 50 % pendant plusieurs semaines.

Solution : fixer le seuil à 0,5

3. Insuffisance d'une seule couche cachée

Avec 1 couche, accuracy bloquée à 72 %. Ajout d'une 2^e couche + activation ReLU → 76,6 %.

Justification : un perceptron à 1 couche ne peut tracer que des frontières linéaires, or les patterns d'attaque sont non-linéaires.

4. Problème du learning rate decay

La courbe d'apprentissage initiale divisait lr par 2 toutes les 10 itérations → lr tend vers 0 en ~60 itérations → loss bloquée à zéro à cause de l'absence de mise à jour.

Solution : passer à Adam, qui adapte le pas par paramètre automatiquement.

Implémentation de la méthode ADAM

2 idées combinées : le momentum et le pas adaptatif par paramètre

```
def update(self, grads):
    self.t += 1
    for name in ['W1', 'b1', 'W2', 'b2', 'W3', 'b3']:
        # Limite sur les gradients pour éviter l'explosion des gradients
        g = np.clip(grads[name], -5.0, 5.0)
        # Récupération de la direction moyenne en intégrant le gradient courant
        self.m[name] = self.betal * self.m[name] + (1 - self.betal) * g
        # Récupération de la dispersion moyenne en intégrant le gradient courant
        self.v[name] = self.beta2 * self.v[name] + (1 - self.beta2) * (g ** 2)

        # Réajustement, car au début m et v sont initialisé à 0
        # Correction en donnant plus de poids aux premiers gradients puis diminution
        m_hat = self.m[name] / (1 - self.betal ** self.t)
        v_hat = self.v[name] / (1 - self.beta2 ** self.t)
        param = getattr(self, name)
        # pas adaptatif = (direction moyenne) / (racine de la variance)
        param -= self.lr * m_hat / (np.sqrt(v_hat) + self.adam_eps)
        setattr(self, name, param)
```

Le contexte technique

Le clipping

Les 2 moments

La correction de biais

La mise à jour finale

Conclusion

Conclusion

Points d'amélioration pour une v2

A rajouter

Des tests unitaires
dès le début du projet

Ingestion du flux
via tcpdump

A renforcer

Un pilotage de projet
plus structuré

Connaissance antérieure des
concepts abordés